Threadly — boutique en ligne sur AWS

Projet AWS · Infrastructures cloud et virtualisation
Enide Djender · Baptiste Bertrand · Mika Vivier

Trois solutions — **serverless**, **machines virtuelles** et **hybride** — décrites en Terraform, région eu-west-3 (Paris), testées en local (Docker) puis sur AWS.

Table des matières

Corps du rapport (pages 2 à 5)

1. Contexte et exigences
2. Les trois architectures
3. Un code métier, trois infrastructures
4. Free Tier et coûts
5. Sécurité
6. Résultats
7. Conclusion et limites

Annexes

- A. Fonctionnalités et services AWS utilisés
- B. Schémas d'architecture en grand format
- C. Captures
- D. Sorties détaillées des tests et de k6
- E. Déroulement du projet, bugs trouvés et incident CloudTrail
- F. Coûts et sécurité en détail
- G. Présenté mais non implémenté
- H. Mode d'emploi pour le correcteur

1. Contexte et exigences

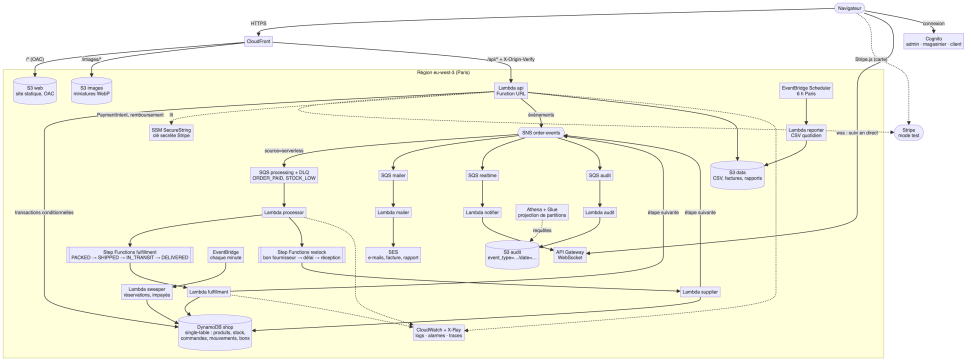
Threadly est une boutique de vêtements fictive (t-shirts, pulls, baskets). Les clients parcourent et recherchent le catalogue, créent un compte, commandent, paient par carte (Stripe en mode test), suivent leur commande en temps réel jusqu'à la livraison et peuvent la retourner sous 14 jours. En interne, chaque événement est archivé pour l'audit, la commande passe par une préparation simulée (emballage, expédition, transit, livraison), le stock est géré par entrepôt avec réapprovisionnement automatique, et un back-office fournit tableau de bord, retours et rapport quotidien (liste complète : annexe A).

Les trois solutions sont **déployées ensemble** par un seul `./up.sh` et supprimées par `./down.sh`. Elles servent le même front, partagent le bus d'événements et l'audit, et passent les mêmes **36 scénarios de test**, ce qui permet de les comparer directement.

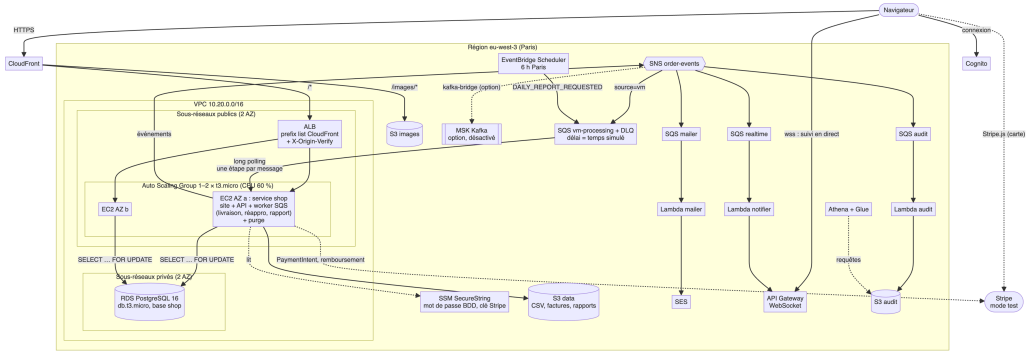
Exigence du sujet	Serverless	VM	Hybride
Catalogue via une interface web	CloudFront + S3 (OAC) ; Lambda `api` + DynamoDB	CloudFront → ALB → EC2 → RDS PostgreSQL	CloudFront + S3 ; Lambda `hybrid-read` + DynamoDB
Commander un ou plusieurs produits	Transaction DynamoDB conditionnelle	Transaction SQL `SELECT … FOR UPDATE`	EC2 → transaction SQL (RDS)
Message de confirmation	SES (Lambda `mailer`) : confirmation avec facture PDF, expédition, livraison… ; suivi en direct (WebSocket API Gateway)	identique	identique
Enregistrer les commandes (audit, reporting)	SNS → SQS → Lambda `audit` → S3 partitionné, requêtable par Athena	identique	identique
Traitement interne	Step Functions (Lambdas `fulfillment`, `supplier`)	Worker SQS sur les EC2	Step Functions ; réappro par l'EC2
Serverless **et** Compute EC2	Lambda, DynamoDB, Step Functions	VPC, ALB, ASG EC2, RDS	les deux (lectures Lambda, écritures EC2)
Événementiel, streaming	SNS + SQS filtrés	SNS + SQS ; MSK en option	SNS + SQS ; projection CQRS
Terraform, haute disponibilité	`infra/`, services multi-AZ	ALB et ASG sur 2 AZ	lectures multi-AZ, écritures sur 2 AZ
Coût nul (Free Tier)	§4	§4	§4

2. Les trois architectures

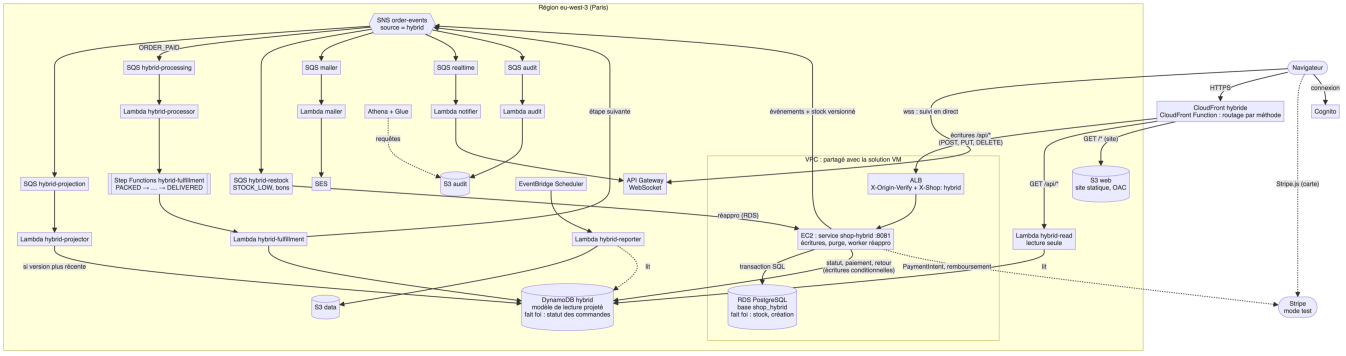
Schémas en grand format : annexe B.



Serverless. CloudFront sert le site depuis S3 et transmet `/api/*` à la Function URL de la Lambda `api` (en-tête secret d'origine). Les données sont dans une table DynamoDB (*single-table design*), modifiées par transactions conditionnelles. Le traitement est orchestré par **Step Functions** : livraison (`fulfillment`) et réappro (`restock`). Rien ne tourne sans requête : coût nul au repos.



Machines virtuelles. CloudFront → ALB (2 AZ, n'accepte que CloudFront) → Auto Scaling Group de `t3.micro`. Chaque instance sert le site et l'API, et fait **tout le traitement interne sur EC2** : un worker lit sa file SQS, une étape par message, le délai de la file simulant le temps passé. Les données sont dans RDS PostgreSQL, en sous-réseau privé.



Hybride. Chaque partie va sur le service qui lui convient : site sur S3, lectures sur Lambda + DynamoDB, écritures sur EC2 + RDS (transaction SQL, sans quota Lambda), livraison par Step Functions. Une CloudFront Function route par méthode HTTP. **RDS fait foi pour le stock et la création** ; DynamoDB est un **modèle de lecture** mis à jour par événements versionnés (CQRS, cohérence à terme) et fait foi pour le statut des commandes. L'hybride réutilise le VPC, l'ALB, l'EC2 et l'instance RDS de la VM : il n'ajoute **aucune ressource facturée à l'heure**.

3. Un code métier, trois infrastructures

Pourquoi partager le code. Les règles (stock, paiement, livraison, retours, e-mails, facture, tableau de bord) sont écrites une seule fois dans le paquet `src/shop`, utilisé tel quel par les Lambdas et par `app.py` sur les EC2. Seul l'accès aux données change (`DynamoStore` ou `SqlStore`, même contrat). Une règle corrigée l'est pour les trois, les 36 scénarios s'appliquent aux trois, et **les différences mesurées (§6) viennent de l'infrastructure, pas du code**.

Dimension	Serverless	VM	Hybride
Calcul	Lambda ; Step Functions	EC2 dans un ASG (1 à 2), service systemd : API, worker, purge	Lambda (lectures), Step Functions ; EC2 (écritures, réappro)
Données	DynamoDB, transactions conditionnelles	RDS PostgreSQL, transactions SQL	RDS (stock, création) + DynamoDB (statut, lecture)
Réseau	Pas de VPC ; Function URL derrière CloudFront	VPC 2 AZ, ALB limité à CloudFront, RDS privé	Les deux, routage par méthode
Montée en charge	Automatique, mais quota de 10 Lambdas simultanées	ASG sur le CPU ; pool de 10 connexions	Lectures élastiques, écritures sur l'EC2
Exploitation	Rien à patcher, traces X-Ray par fonction	AMI, user data, systemd, Session Manager	Les deux, plus la cohérence de deux modèles
Coût au repos	0 $	≈ 0,07 $/h (ALB, EC2, RDS, IPv4)	0 $ de plus que la VM (ressources partagées)

Orchestration : Step Functions contre worker sur VM. La VM reste une vraie solution « machines virtuelles » : son traitement tourne sur EC2, sans Step Functions. Les deux approches appliquent les mêmes transitions (`domain.TRANSITIONS`), par mises à jour conditionnelles :

	Step Functions (serverless, hybride)	Worker SQS sur EC2 (VM)
Processus	Explicite et versionné (fulfillment.asl.json) : `Task`, `Wait`, `Choice`	Implicite : chaque étape publie l'événement suivant
Attente	État `Wait`, rien de facturé pendant l'attente	Délai de livraison de la file SQS (15 min au plus)
Erreurs, doublons	`Retry` déclaratif ; nom d'exécution = commande	3 réceptions puis DLQ ; transition conditionnelle
Observabilité	Historique graphique, traces X-Ray	Journaux du service, métriques des files
Coût	9 transitions par commande, 4 000 gratuites par mois	Inclus dans l'EC2 déjà payée

Step Functions rend le processus visible et rejouable sans serveur ; c'est le bon choix dès qu'il y a des branches, de longues attentes ou des compensations. Le worker est plus simple et ne coûte rien de plus quand une instance tourne déjà, mais l'enchaînement est dispersé dans le code.

4. Free Tier et coûts

L'infrastructure n'existe que pendant les sessions de travail (`up.sh` puis `down.sh`). Le compte, créé en octobre 2026, dépend du **plan gratuit à crédits** (6 mois, 200 $ de crédits au plus) : ce qui n'est pas gratuit à vie est payé par les crédits. Détail par service et choix de dimensionnement : annexe F.

Poste	Coût d'une heure de démo (eu-west-3, prix public)
ALB + EC2 `t3.micro` + RDS `db.t3.micro` + IPv4 publiques (VM, partagés par l'hybride)	≈ 0,07 $/h
Lambda, SNS, SQS, S3, CloudFront, Step Functions, Scheduler, SES, X-Ray, Cognito ; DynamoDB facturé à la requête	≈ 0 $ (offre toujours gratuite ou quelques centimes, volumes de test)
Athena (10 Mo facturés au minimum par requête)	≈ 0,00005 $ par requête
Désactivé par défaut : MSK (≈ 0,10 $/h par broker), NAT Gateway, RDS Multi-AZ, KMS client, Route 53	0 $
Total pour les trois solutions	**≈ 0,07 $/h, soit moins de 2 $ pour 24 h**

Un budget AWS (1 $/mois) alerte par e-mail ; toutes les ressources portent le tag `Project = threadly`. Le test de charge (§6) a coûté moins d'un centime.

5. Sécurité

- **Comptes : Amazon Cognito** (groupes `admin`, `magasinier` ; tout compte connecté est client). L'API vérifie elle-même chaque jeton : RS256 uniquement, signature avec la clé du JWKS de Cognito, émetteur, client et expiration ; un émetteur non Cognito est toujours refusé sur AWS. Chaque route vérifie le rôle (anonyme 401, mauvais rôle 403).
- **Paiement et PCI DSS** : la carte est saisie dans Stripe.js (iframe Stripe) et ne transite jamais par nos serveurs, ce qui place la boutique dans le périmètre le plus réduit (questionnaire SAQ A). Le serveur relit le paiement chez Stripe (montant, devise, commande) avant de confirmer, avec des clés d'idempotence pour ne jamais créer de doublon.
- **Secrets** : clé secrète Stripe et mot de passe RDS dans **SSM Parameter Store** (SecureString), lus à l'exécution ; `terraform.tfvars` et `terraform.tfstate` ignorés par git ; aucun secret dans le code ni dans le dépôt.
- **IAM au moindre privilège** : un rôle par Lambda et un pour les EC2, actions et ARN précis (la Lambda de lecture hybride n'a aucun droit d'écriture) ; les files SQS n'acceptent que le topic SNS ; Grafana local lit la base avec un compte en lecture seule.
- **Réseau et données** : RDS privé, ALB limité à CloudFront, aucun port SSH (Session Manager), IMDSv2 ; HTTPS imposé ; chiffrement au repos partout ; CloudTrail actif.

Détail (tableau des rôles IAM, en-têtes secrets, chiffrement) : annexe F ; e-mails reçus et paiements dans le tableau de bord Stripe : annexe C.7.

6. Résultats

Tests fonctionnels. `scripts/test_scenario.sh` joue 36 scénarios par l'API : catalogue, commandes concurrentes, stock et réservations, comptes et rôles, paiement Stripe, workflow jusqu'à `DELIVERED`, facture, e-mails, retours, réappro, tableau

de bord, rapport quotidien, Athena, X-Ray, Grafana (liste : annexe D).

Session	Serverless	VM	Hybride
Local (Docker), deux passages de suite sans réinitialisation	36/36 ✅ aux deux passages	36/36 ✅ aux deux passages	36/36 ✅ aux deux passages
AWS, session finale du 28/10/2026 (les trois en même temps)	36/36 ✅ (scénario 35 rejoué, annexe D)	36/36 ✅	36/36 ✅

Le scénario 4 de la comparaison ci-dessous vient du premier passage de la session : rejoué sur un déploiement déjà testé, il porte sur un produit de test d'une seule unité (6 commandes seulement, annexe D). Les scénarios propres à un environnement (Athena et X-Ray sur AWS, Grafana en local) sont comptés comme réussis « sans objet » ailleurs (annexe D). La session finale s'est déroulée sur un second compte (le quota SES du premier était épuisé, annexe E), dont les quotas sont **identiques** : 10 exécutions Lambda simultanées, SES en bac à sable (200 e-mails par jour, 1 par seconde). Les résultats du scénario 4 et de k6 sont donc directement comparables à ceux des sessions précédentes. Toute la session a envoyé **96 e-mails** par SES, sous la limite de 200 par jour, contre plus de 270 avant que les commandes de test (adresses `example.com`) ne soient plus envoyées (annexe E, `captures/ses_envois.txt`).

Test de charge k6 (8 utilisateurs virtuels, 60 s par solution, parcours de lecture, solutions testées l'une après l'autre) :

Solution	Requêtes	Req/s	Médiane	p95	Max	Réponses attendues	429	5xx
Serverless	1068	13,8	52 ms	272 ms	514 ms	100,0 %	0	0
VM	1110	14,5	56 ms	241 ms	390 ms	100,0 %	0	0
Hybride	1074	14,1	47 ms	251 ms	1 110 ms	100,0 %	0	0

Comparaison chiffrée (session AWS finale) :

	Serverless	VM	Hybride
Commande payée → `DELIVERED` (4 étapes, 5 s simulées chacune)	18 s	22 s	18 s
23 commandes simultanées (scénario 4, premier passage)	10 acceptées, **13 refusées par Lambda (429)** ; stock final 8 = 18 − 10	18 acceptées, 5 refusées (stock épuisé)	18 acceptées, 5 refusées (stock épuisé)
Latence de lecture p95 (k6)	272 ms (médiane 52 ms)	241 ms (médiane 56 ms)	251 ms (médiane 47 ms)
Cohérence du stock affiché	immédiate	immédiate	à terme (moins d'une seconde)
Si RDS tombe	non concerné	boutique arrêtée	catalogue et suivi servis, commandes refusées

7. Conclusion et limites

Les trois solutions rendent le même service avec le même code ; leurs différences sont celles de l'infrastructure. **Le serverless** est le plus simple et ne coûte rien au repos, mais le quota de 10 exécutions Lambda simultanées des comptes récents limite les rafales : 13 commandes sur 23 refusées en 429 quand elles arrivent au même instant (scénario 4), alors qu'un trafic régulier de 14 requêtes par seconde passe sans refus (k6). **La VM** tient les écritures et a la meilleure latence à faible charge, au prix d'un coût horaire et d'une exploitation plus lourde. **L'hybride** combine des écritures transactionnelles sans quota et des lectures sans serveur, et reste partiellement disponible si RDS tombe ; il est le plus complexe (deux modèles à garder cohérents). Pour une boutique de cette taille, le serverless suffit ; l'hybride se justifie quand les écritures doivent tenir des pics.

Limites (et améliorations proposées) : état Terraform local, sans verrou (backend S3 avec verrou) ; quota Lambda (demande d'augmentation, *reserved concurrency*) ; RDS Single-AZ sans sauvegarde (Multi-AZ en production) ; pas de webhook Stripe (à ajouter en production, annexe G) ; SES en *sandbox*, limité à 200 e-mails par jour (demande d'accès en production) ; Grafana seulement en local. Les services étudiés mais non déployés (Bedrock, Personalize, Forecast, WAF, QuickSight, Secrets Manager, OpenSearch, ERP) sont présentés en annexe G.

Annexe A. Fonctionnalités et services AWS utilisés

A.1 Fonctionnalités, par thème

Thème	Fonctionnalités
Catalogue et stock	Catalogue avec recherche, filtres et tris ; stock par entrepôt (Paris, Lyon) ; réservations de panier avec expiration ; mouvements (réappro, casse, inventaire, retour) et journal ; alertes de stock bas ; valorisation ; import et export CSV par S3 ; images produit (dépôt présigné, miniatures WebP par Lambda)
Comptes et clients	Comptes Cognito (admin, magasinier, client) ; profil, « mes commandes » ; avis réservés aux acheteurs ; liste de souhaits avec alerte de retour en stock (compte et e-mail) ; codes promo
Commande et paiement	Commande d'un ou plusieurs produits ; paiement Stripe en mode test, avec purge des impayés ; frais de port (standard, express) ; annulation avant expédition, remboursée ; facture PDF
Livraison et retours	Workflow jusqu'à `DELIVERED` (Step Functions, ou worker EC2 pour la VM) ; suivi en temps réel par WebSocket ; e-mails SES ; retours sous 14 jours, réceptionnés puis remboursés ; réapprovisionnement fournisseur automatique
Pilotage	Back-office ; tableau de bord ; rapport quotidien (EventBridge Scheduler) ; audit S3 requêtable par Athena et Glue ; traces X-Ray ; Grafana local
Qualité	36 scénarios de test sur les trois solutions (local et AWS) ; test de charge k6 ; intégration continue GitHub Actions ; stack Docker locale identique à AWS

Cycle de vie d'une commande (identique partout, défini dans `shop/domain.py`) : `PENDING_PAYMENT → CONFIRMED → PACKED → SHIPPED → IN_TRANSIT → DELIVERED`, avec `CANCELLED` (avant l'expédition, ou impayée) et `RETURN_REQUESTED → RETURNED`.

Paiement. Le PaymentIntent est créé côté serveur avec une clé d'idempotence (`threadly-pi-<commande>`) ; le navigateur paie avec Stripe.js ; `POST /api/orders/{id}/confirm-payment` relit le paiement chez Stripe. Sans webhook, un client peut payer puis fermer son navigateur : la **purge** relit le PaymentIntent avant toute annulation, confirme la commande s'il a réussi, et sinon annule le PaymentIntent *avant* la commande. Remboursements : clé `threadly-refund-<commande>`. `payment_mode = "simulated"` permet de tout tester sans compte Stripe.

E-mails. SES est en *sandbox* : tous les e-mails vont à l'adresse vérifiée, avec le destinataire prévu en tête du message. L'identité est vérifiée une seule fois, hors Terraform (bloc `check`, contrôle dans `up.sh`) : pas de nouvel e-mail de vérification à chaque déploiement.

Pilotage. Tableau de bord (chiffre d'affaires du jour et des 7 jours, top ventes, ruptures, statuts) ; rapport CSV quotidien (ventes, stock, valorisation) déposé dans S3 et envoyé par SES, à 6 h ou à la demande ; table Athena `events` partitionnée par **projection** (`event_type`, `date`), sans crawler, avec trois requêtes nommées ; traces X-Ray des Lambdas et des machines d'état.

A.2 Services AWS utilisés

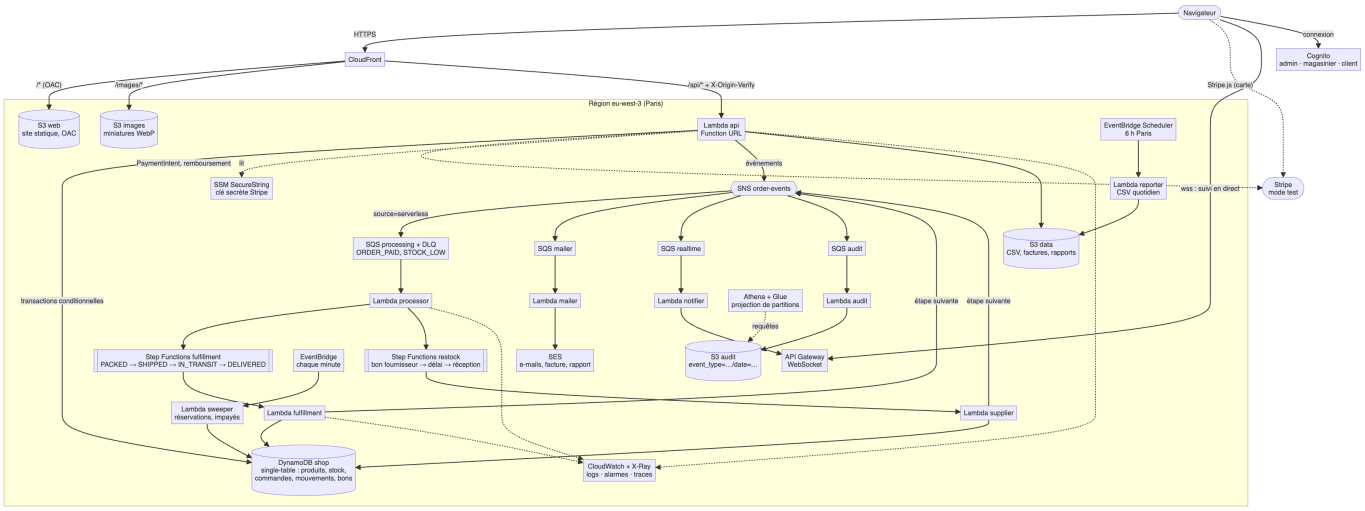
Service	Rôle dans le projet	Solutions
CloudFront (+ CloudFront Function)	Point d'entrée HTTPS, cache du site et des images, routage hybride	toutes
S3	Site, images, audit, données (CSV, factures, rapports), résultats Athena, CloudTrail	toutes
Lambda (Function URL)	API, traitement, purge, audit, e-mails, miniatures, temps réel, rapport	toutes (VM : audit, e-mails, temps réel)
DynamoDB	Données du serverless, modèle de lecture de l'hybride, connexions WebSocket	serverless, hybride
VPC, ALB, EC2, Auto Scaling	Serveurs d'application sur 2 AZ	VM, hybride
RDS PostgreSQL	Base relationnelle (une base par solution)	VM, hybride
SNS, SQS (+ DLQ)	Bus d'événements, files de traitement	toutes
Step Functions	Workflows de livraison et de réappro	serverless, hybride
EventBridge, EventBridge Scheduler	Purge chaque minute ; rapport quotidien	toutes
Cognito	Comptes et rôles	toutes

Service	Rôle dans le projet	Solutions
API Gateway (WebSocket)	Suivi en temps réel	toutes
SES	E-mails clients et équipe	toutes
SSM Parameter Store, Session Manager	Secrets ; accès aux EC2 sans SSH	toutes
Glue Data Catalog, Athena	Requêtes SQL sur l'audit	toutes
CloudWatch, X-Ray, CloudTrail, Budgets	Logs, alarmes, tableau de bord, traces, audit des appels, budget	toutes
IAM, KMS (option)	Rôles au moindre privilège ; clé client en option	toutes
MSK (option, désactivé)	Streaming Kafka	VM

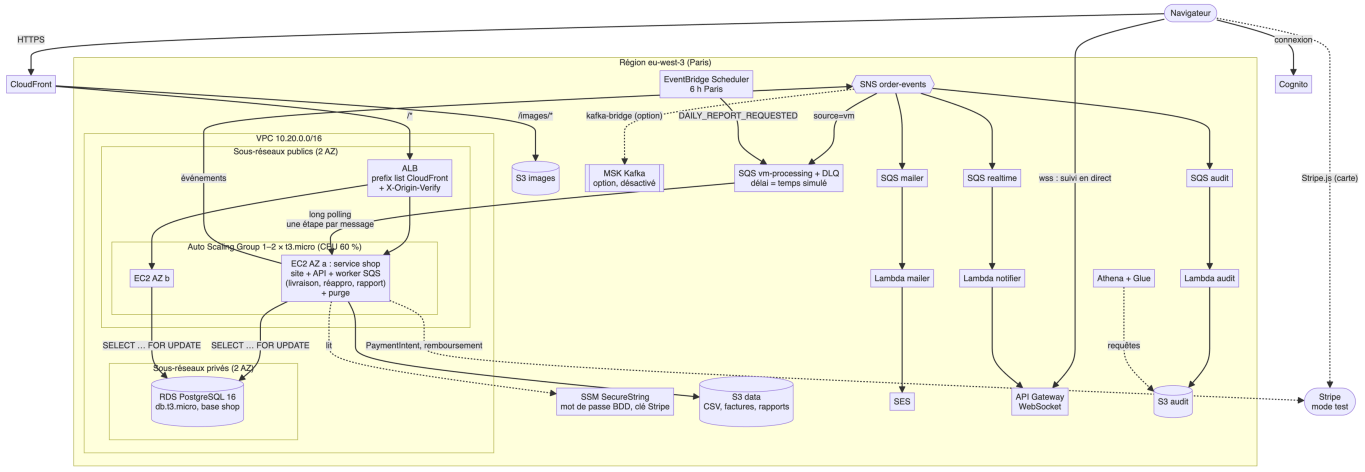
Annexe B. Schémas d'architecture en grand format

Sources Mermaid : `architecture/` (`serverless.mmd`, `vm.mmd`, `hybrid.mmd`).

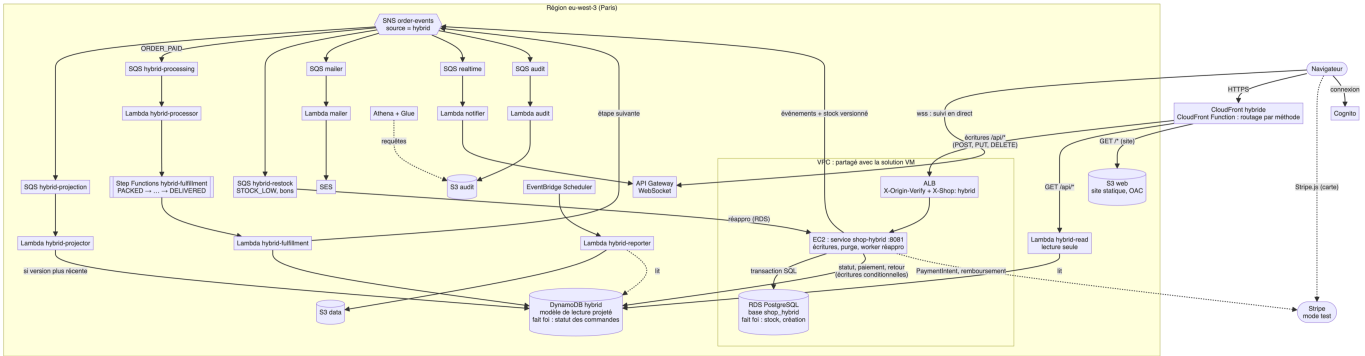
B.1 Serverless



B.2 Machines virtuelles



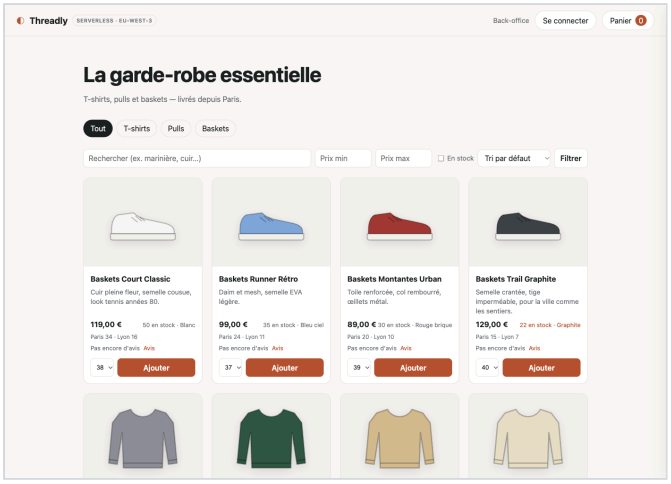
B.3 Hybride



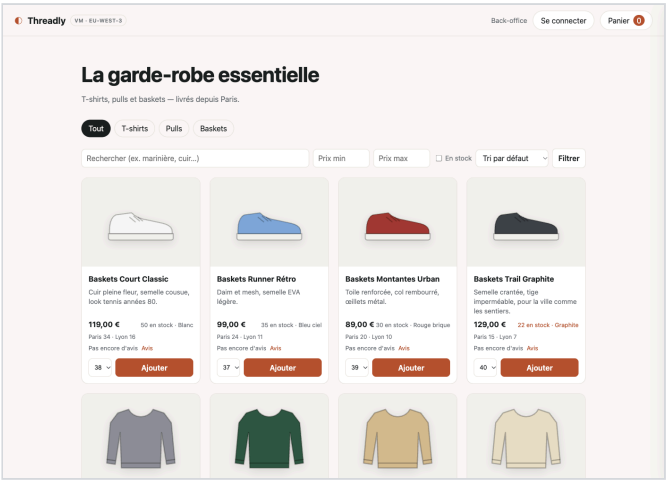
Annexe C. Captures

Toutes les captures datent de la session AWS finale du 28/10/2026, sauf Grafana et le tableau de bord du back-office, pris sur la stack locale. Les graphiques CloudWatch sont produits par l'API `GetMetricWidgetImage` ; Step Functions, X-Ray et Athena sont restitués à partir de leurs API (historique d'exécution, traces, résultats de requêtes).

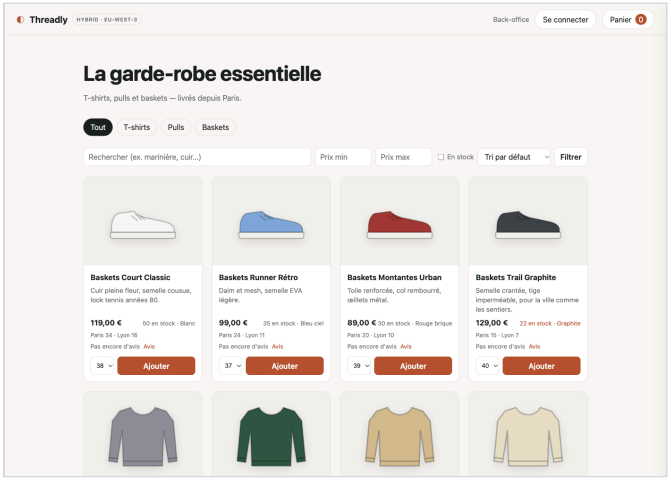
C.1 Front des trois solutions (session AWS finale)



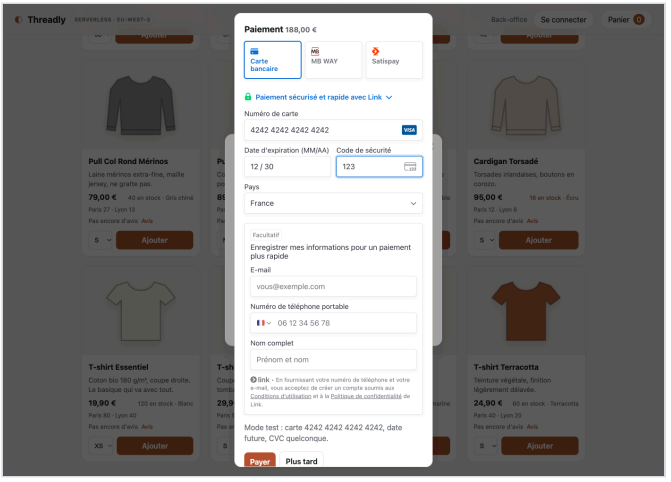
Serverless : catalogue



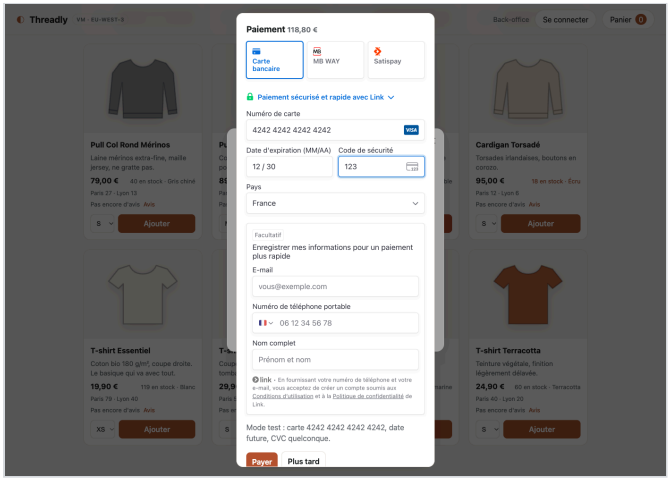
VM : catalogue



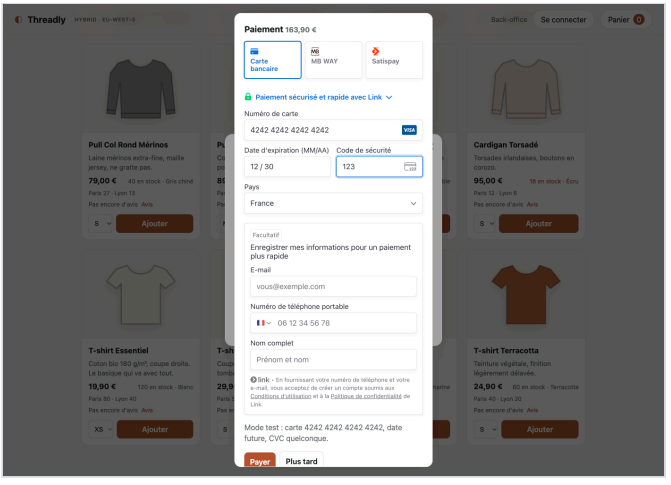
Hybride : catalogue



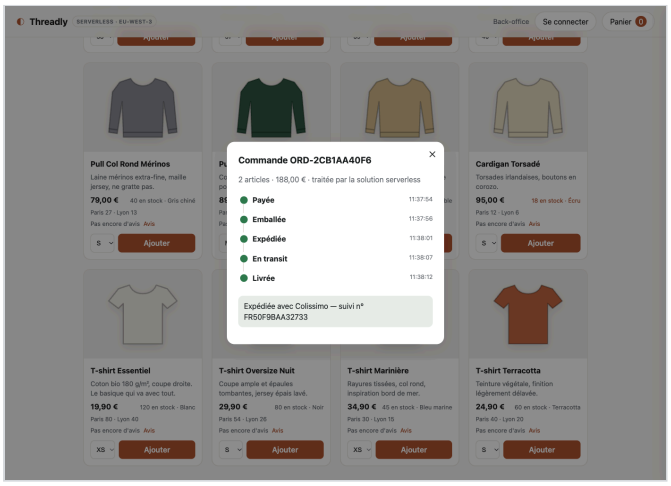
Serverless : paiement par carte (Stripe Elements, carte de test 4242)



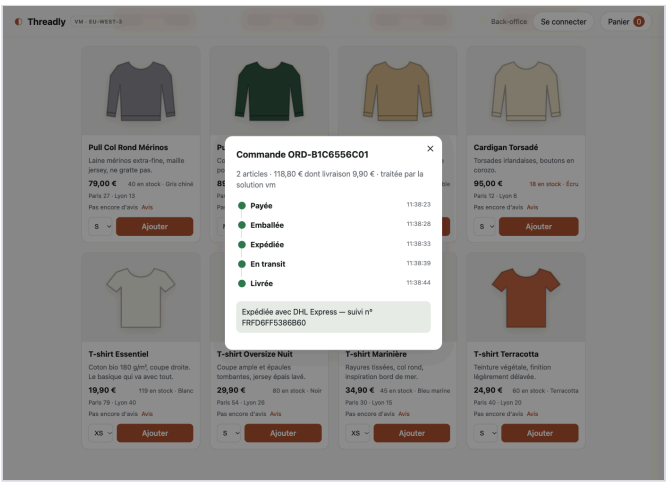
VM : paiement par carte (Stripe Elements, carte de test 4242)



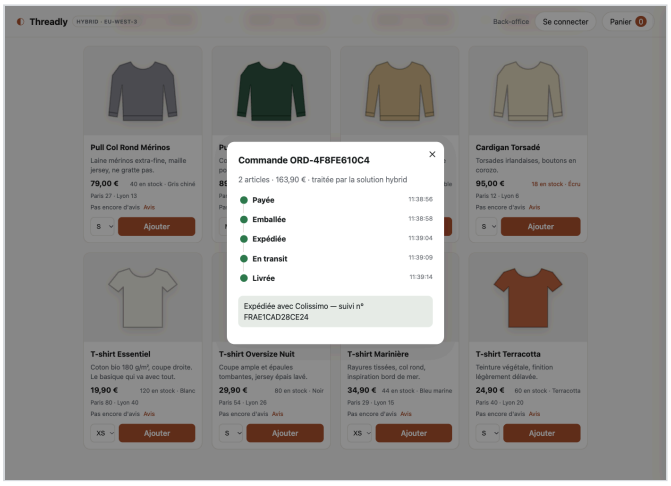
Hybride : paiement par carte (Stripe Elements, carte de test 4242)



Serverless : commande suivie jusqu'à la livraison



VM : commande suivie jusqu'à la livraison

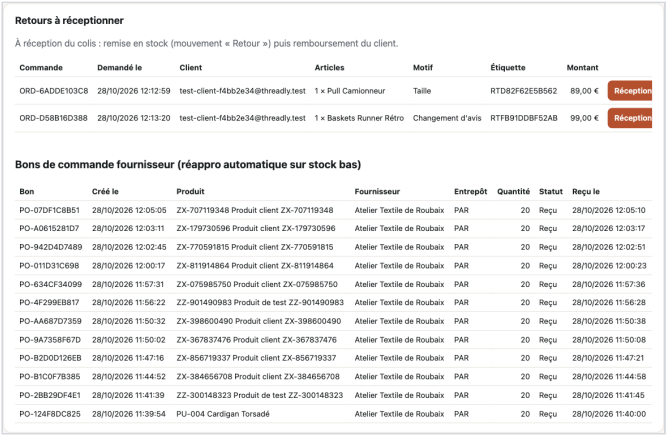


Hybride : commande suivie jusqu'à la livraison

C.2 Back-office



Tableau de bord (admin) : chiffre d'affaires, top ventes, ruptures, statuts (stack locale, solution VM)

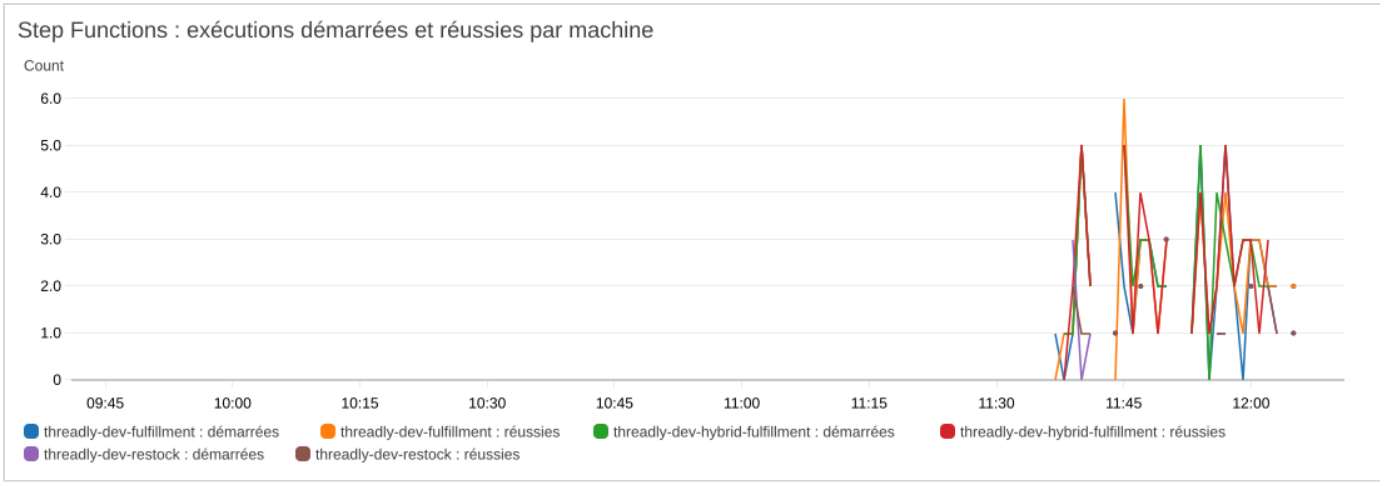


Retours à réceptionner et bons de commande fournisseur

C.3 Step Functions

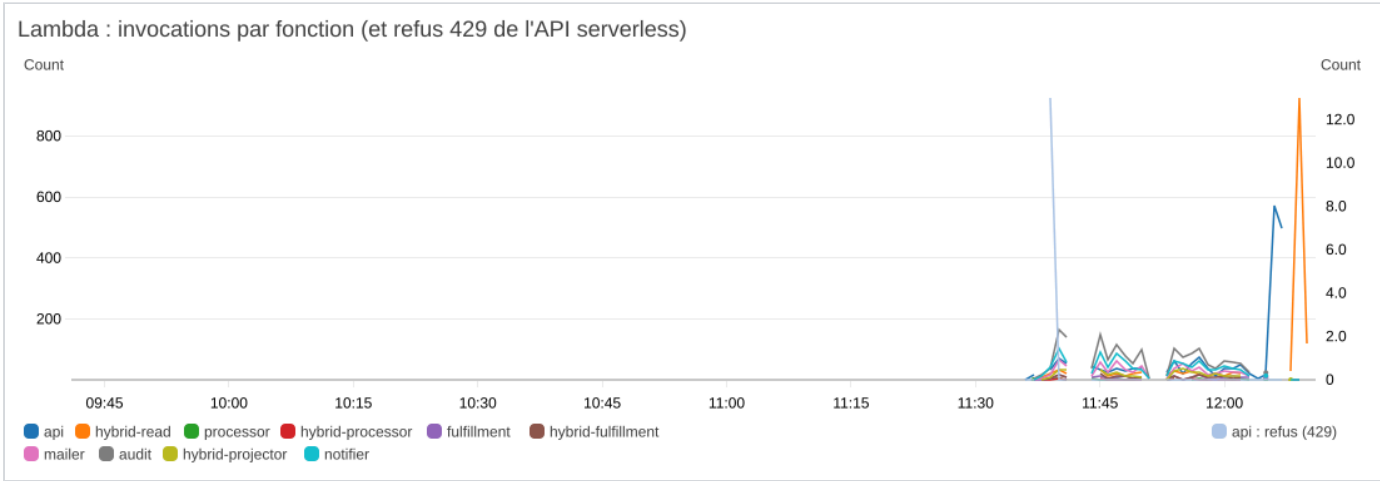


Exécution réussie de la machine fulfillment (historique de l'exécution, délais réels)

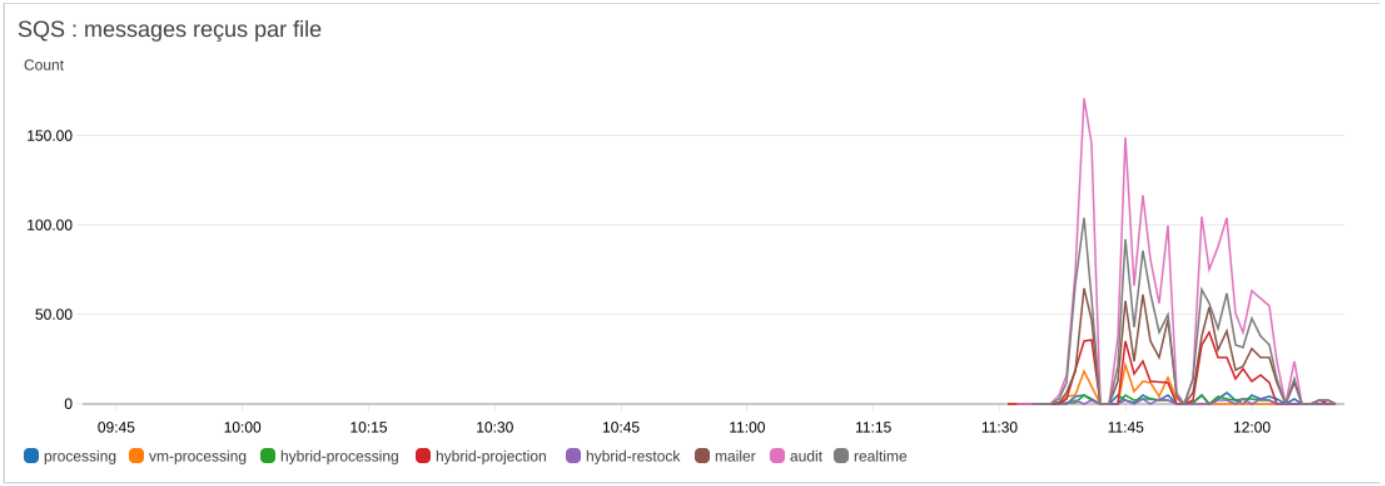


Exécutions démarrées et réussies par machine (CloudWatch)

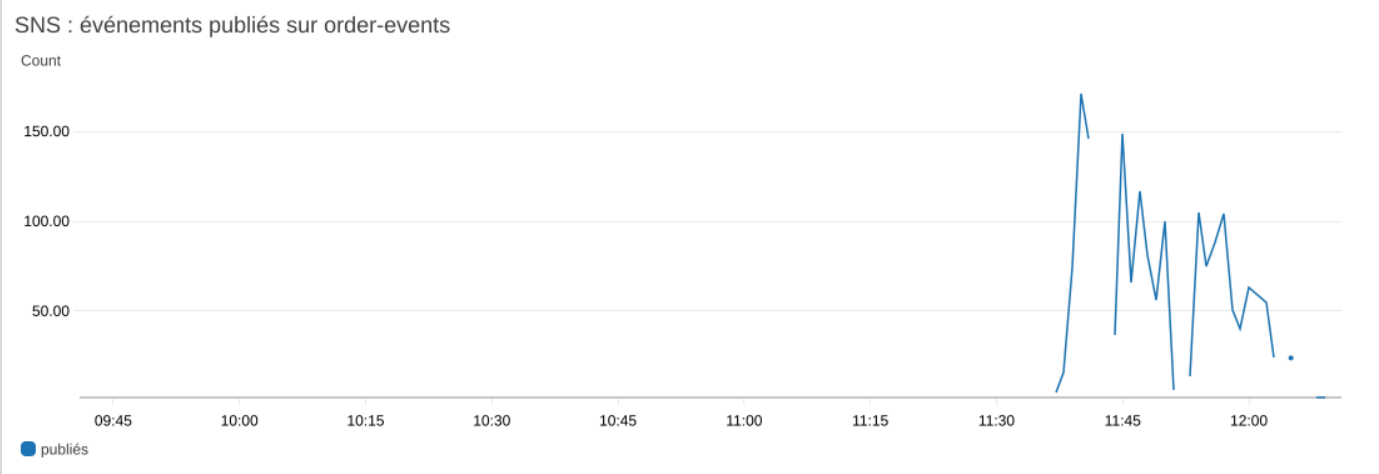
C.4 CloudWatch (heures de Paris)



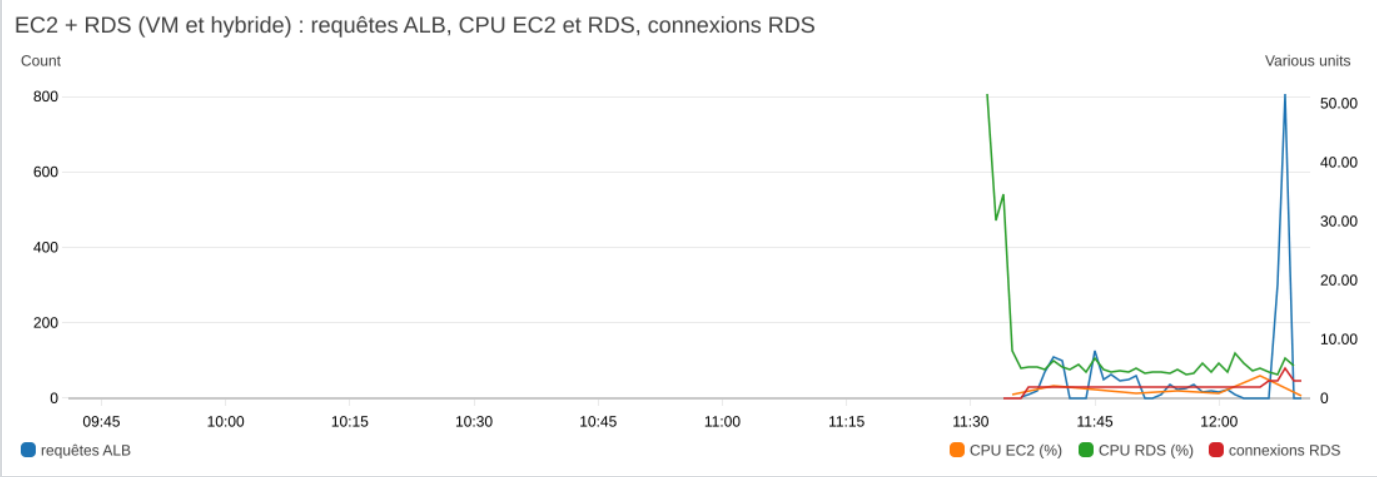
Invocations des Lambdas, et refus 429 de l'API serverless



Messages reçus par file SQS

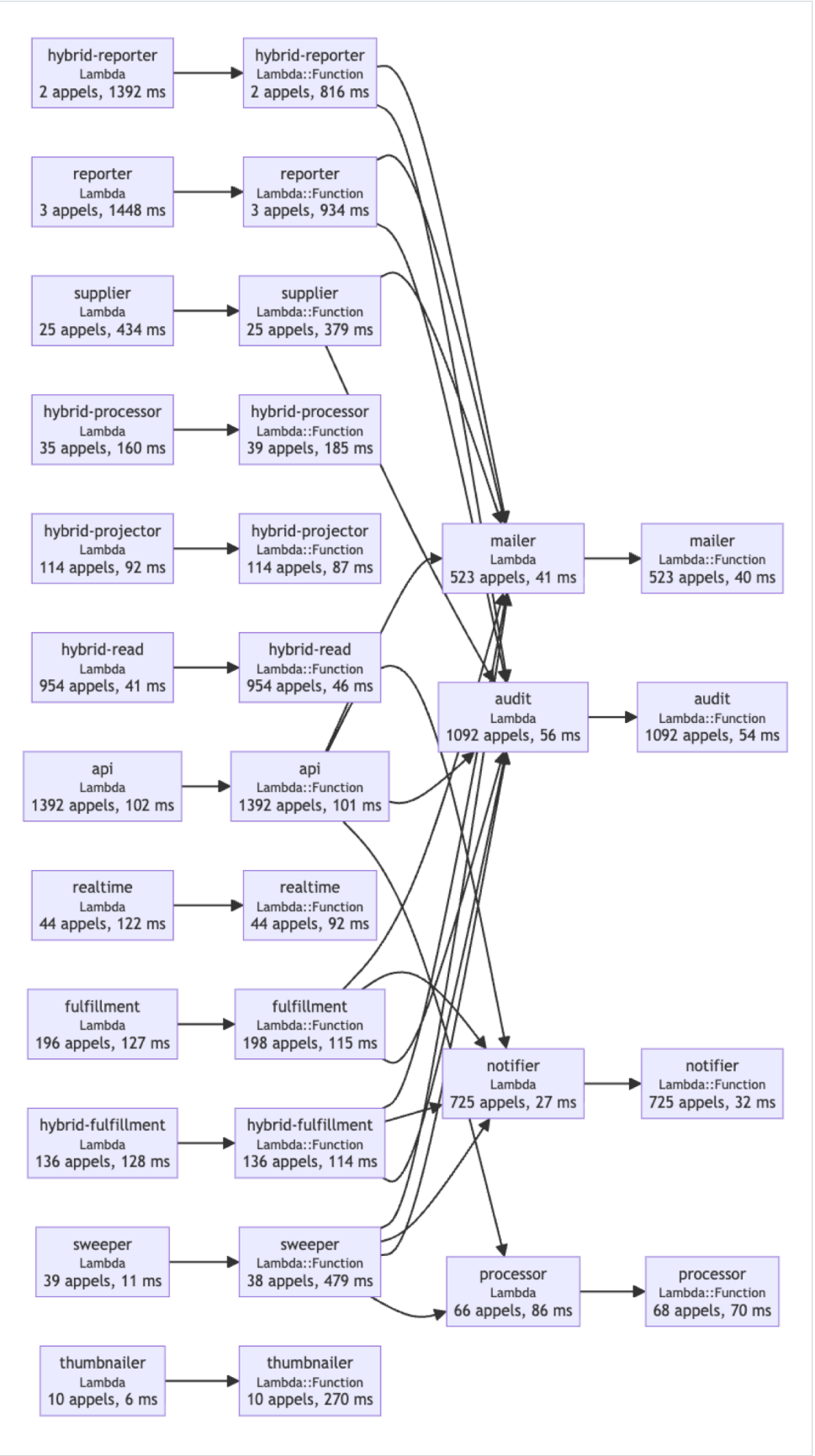


Événements publiés sur SNS



EC2 + RDS (VM et hybride) : requêtes ALB, CPU, connexions

C.5 X-Ray



Carte des services tracés (appels et temps moyen par service)

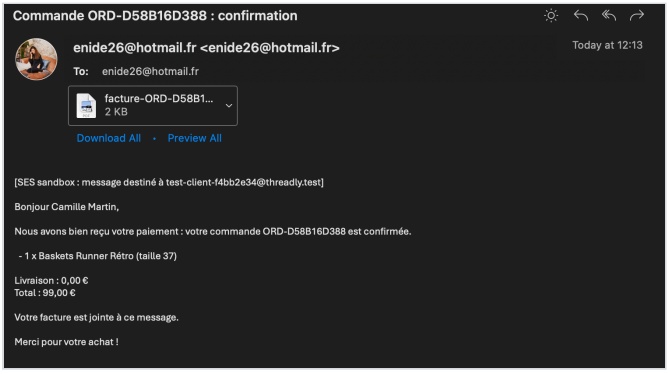
C.6 Grafana (stack locale)



Tableau de bord « ventes et stock » provisionné, base de la VM

C.7 E-mails et paiement (boîte de réception, tableau de bord Stripe)

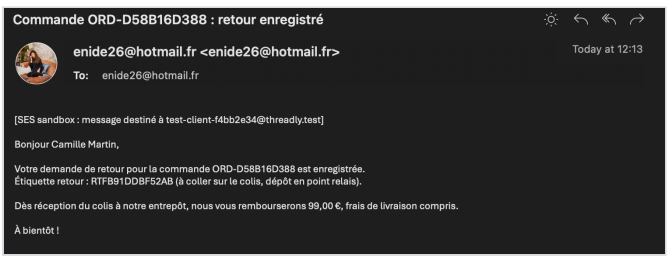
E-mails réellement reçus pendant la session finale, envoyés par la Lambda `mailer` via SES (bac à sable : tous redirigés vers l'adresse vérifiée, le destinataire prévu en tête du message), et paiements de test enregistrés par Stripe.



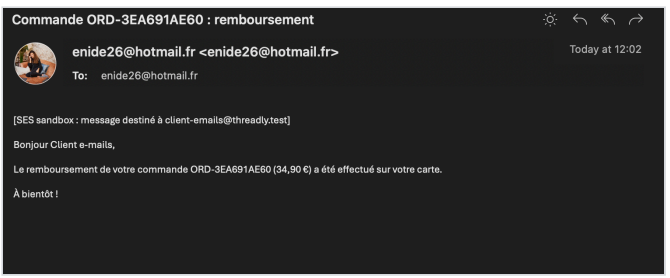
Confirmation de commande, facture PDF jointe



Expédition : transporteur et numéro de suivi



Retour enregistré : étiquette retour



Remboursement après réception du retour



Alerte de retour en stock (liste de souhaits)



Alerte de stock bas (équipe)



Bon de commande fournisseur (réappro automatique)



Rapport quotidien, CSV joint

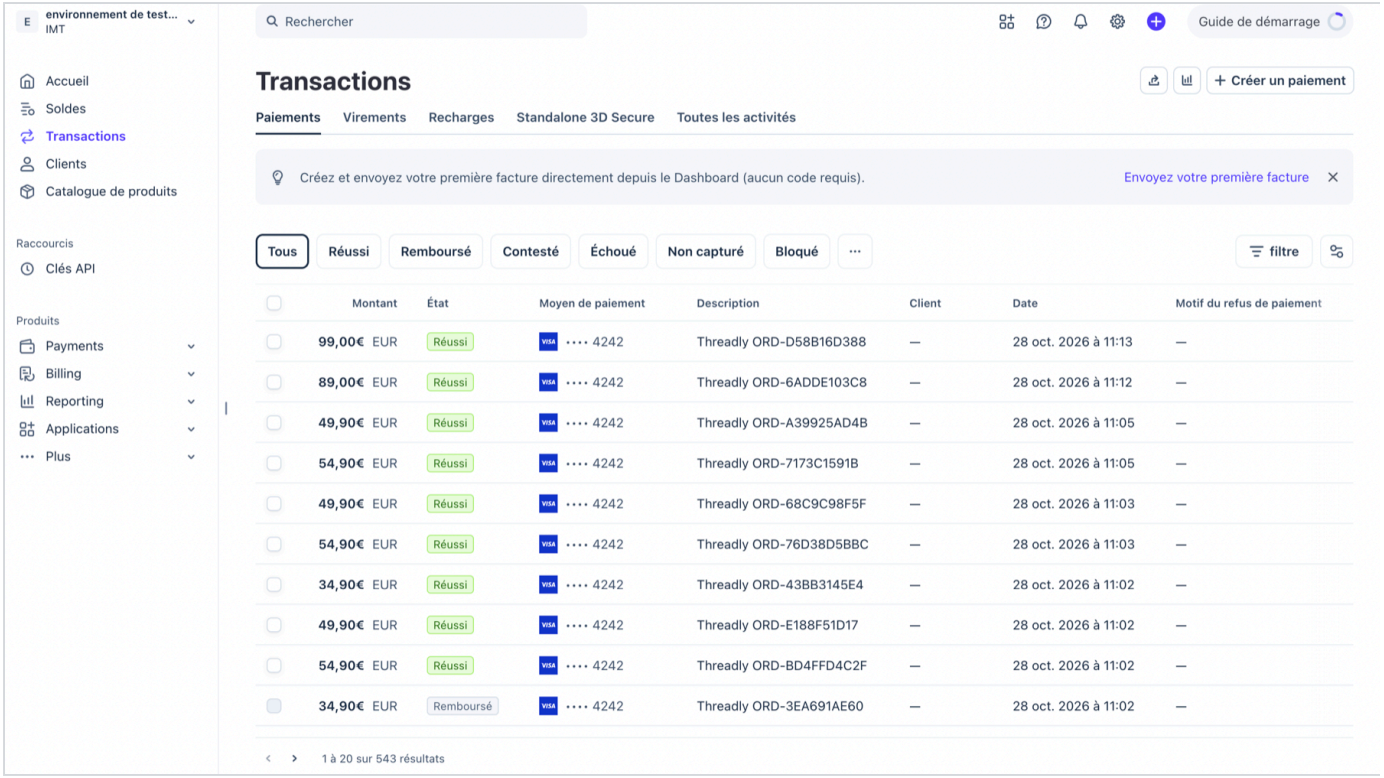


Tableau de bord Stripe (mode test) : paiements « Threadly ORD-… » par carte de test 4242, réussis, et un remboursement

C.8 Captures textuelles (API AWS)

Fichier	Contenu
`captures/step_functions_executions.txt`	Step Functions : historique des dernières exécutions
`captures/xray_services.txt`	X-Ray : services et traces
`captures/athena_resultats.txt`	Athena : les trois requêtes nommées et leurs résultats
`captures/audit_s3_liste.txt`	Audit S3 : nombre de fichiers par type et liste complète
`captures/audit_s3_exemple.txt`	Audit S3 : un événement complet
`captures/ec2_rds_ressources.txt`	EC2 et RDS : relevés de mémoire et de connexions
`captures/ses_envois.txt`	SES : e-mails envoyés par la session finale (96), la première tentative (270) et chaque passage local

Extrait de `captures/athena_resultats.txt` (chiffre d'affaires par jour et délai commande → livraison) :

```
delai-commande-livraison
État : SUCCEEDED, données lues : 435.6 Ko, durée : 1387 ms

  source     | commandes_livrees | delai_moyen_s
  -----------+-------------------+--------------
  hybrid     | 47                | 19.4
  serverless | 49                | 21.6
  vm         | 24                | 23.1
```

```
chiffre-affaires-par-jour
État : SUCCEEDED, données lues : 152.3 Ko, durée : 1068 ms

  date       | source     | encaisse           | rembourse          | chiffre_affaires
  -----------+------------+--------------------+--------------------+------------------
  2026-10-28 | hybrid     | 3059.7000000000016 | 299.59999999999997 | 2760.1000000000013
  2026-10-28 | serverless | 3188.6000000000013 | 299.59999999999997 | 2889.000000000001
  2026-10-28 | vm         | 1566.7000000000007 | 149.8              | 1416.9000000000005
```

Annexe D. Sorties détaillées des tests et de k6

D.1 Liste des scénarios

#	Scénario	Ce qu'il vérifie
1	Afficher le catalogue	Le front et l'API lisent la base (DynamoDB ou RDS) via CloudFront
2	Commande de 2 produits payée, suivie jusqu'à `DELIVERED`	Paiement par carte de test, puis toute la chaîne : `PENDING_PAYMENT` → `CONFIRMED` → `PACKED` → `SHIPPED` → `IN_TRANSIT` → `DELIVERED` (Step Functions ou worker EC2), transporteur, n° de suivi, stock décrémenté
3	Commande supérieure au stock	Refus `409`, sans aucune modification du stock
4	Commandes simultanées dépassant le stock	Le décrément est atomique (transaction DynamoDB / `SELECT … FOR UPDATE`) : le stock final vaut exactement `stock initial − commandes acceptées`, sans jamais passer sous 0, et chaque requête reçoit `201` ou `409`. Les `429` (requête refusée par Lambda, quota de concurrence du compte atteint) sont comptés à part, signalés par ⚠, sans faire échouer le scénario
5	Fichiers d'audit S3	Les 6 événements de la commande du scénario 2 (`ORDER_CREATED` à `ORDER_DELIVERED`) sont archivés par la Lambda audit dans `s3://<audit_bucket>/events/`
6	Back-office	Routes admin refusées sans jeton (401) ; création, modification, archivage d'un produit (absent du catalogue, commande refusée)
7	Multi-entrepôts	Une commande de 4 unités est servie par Paris (3) puis Lyon (1)
8	Journal des mouvements	Réappro, casse, inventaire et vente dans l'ordre ; une casse supérieure au stock est refusée (409)
9	Annulation	Le stock est remis ; seconde annulation et annulation d'une commande expédiée refusées (409)
10	Réservations	Le stock réservé par un panier est refusé aux autres ; la commande du panier le consomme ; libération manuelle ; expiration (10 s)
11	Réservations simultanées	9 paniers pour 6 unités : jamais plus de 6 réservées, disponible et réservé cohérents
12	Alerte stock bas	`STOCK_LOW` publié sur SNS une seule fois au franchissement du seuil, archivé dans l'audit S3
13	CSV via S3	Export par lien présigné, modification, dépôt par lien présigné, import (1 produit modifié, 1 créé)
14	Valorisation	Valeur = stock physique × prix de revient, par produit, par entrepôt et au total
15	Comptes et rôles	Client 403 sur l'admin ; magasinier : stock oui, catalogue et promos non ; 6 jetons forgés refusés (401), dont un jeton du fournisseur local ; `infra/` ne référence pas le fournisseur local
16	Recherche et filtres	Texte, catégorie, prix minimum et maximum, en stock, tris
17	Codes promo	Remise en % ou en €, limite d'utilisations, expiration ; 4 commandes simultanées sur un code à usage unique : 1 seule acceptée
18	Avis	Réservés aux clients ayant commandé le produit ; un avis par client ; note moyenne à jour
19	Liste de souhaits	Alerte unique quand un produit souhaité revient en stock
20	Mes commandes	Chaque client ne voit que ses commandes
21	Images produit	Formulaire de dépôt présigné réservé à l'admin (type imposé par S3), miniatures WebP 160 et 480 px par la Lambda thumbnailer, faux fichier ignoré, original non public
22	Temps réel (AWS)	WebSocket : statuts de `ORDER_PAID` à `ORDER_DELIVERED` poussés sans polling, alerte de retour en stock poussée au client connecté, jeton forgé refusé ; non applicable en local
23	Paiement par carte	402 sans paiement, carte refusée (`card_declined`), paiement vérifié côté serveur (statut, montant, devise, commande), confirmation rejouée sans effet, `client_secret` jamais exposé par `GET` ; en local : un seul PaymentIntent par commande (clé d'idempotence)
24	Purge des impayés	Commande payée chez Stripe sans retour sur la boutique : confirmée par la purge ; commande jamais payée : PaymentIntent annulé chez Stripe puis commande annulée, stock rendu
25	Frais de port	Standard 4,90 € (offerte dès 60 € d'articles après remise), express 9,90 €, mode inconnu refusé
26	Workflow de livraison	5 étapes jusqu'à `DELIVERED` (Step Functions en serverless et hybride, worker EC2 pour la VM) ; commande payée puis annulée : remboursée une fois, le workflow s'arrête

#	Scénario	Ce qu'il vérifie
27	Facture PDF	Réservée au client de la commande (et à l'admin), une fois payée : anonyme 401, impayée 409, autre client 403 ; PDF valide par lien présigné
28	E-mails SES	Confirmation (facture jointe), expédition, livraison et alerte de retour en stock (liste de souhaits) envoyées ; LocalStack en local, journaux de la Lambda `mailer` sur AWS
29	Retour et remboursement	Commande livrée : retour demandé par son client (anonyme 401, autre client 403, motif inconnu 400, seconde demande 409), réceptionné par le magasinier : `RETURNED`, mouvement `RETURN`, stock remis, remboursée une fois (seconde réception 409), e-mails « retour enregistré » et « remboursement »
30	Retour refusé	Commande non livrée : 409 ; règle des 14 jours après la livraison (13 jours : accepté, 15 jours : refusé)
31	Réappro fournisseur	Deux passages sous le seuil : un seul bon de commande, livré après le délai fournisseur (mouvement `RESTOCK` réf. du bon), stock remonté une fois ; Step Functions en serverless, worker EC2 pour la VM et l'hybride
32	Tableau de bord	Back-office (admin, magasinier ; anonyme 401, client 403) : chiffre d'affaires du jour et des 7 jours, top ventes, ruptures et stocks bas, commandes par statut, mis à jour par une commande
33	Rapport quotidien	CSV (ventes 24 h, stock, valorisation) dans S3 et envoyé par SES, CSV joint ; à la demande (admin seulement) et par la cible d'EventBridge Scheduler (Lambda `reporter`, ou worker EC2 pour la VM)
34	Athena + Glue	Table `events` partitionnée par projection (event_type, date) ; 3 requêtes nommées : chiffre d'affaires par jour, délai commande → livraison, annulations (AWS seulement)
35	X-Ray	Traces des Lambdas et des machines Step Functions (AWS seulement)
36	Grafana (local)	Tableau de bord provisionné, source PostgreSQL en lecture seule : le chiffre d'affaires du jour lu par Grafana est celui du back-office (VM)

D.2 Sorties complètes

Fichier	Contenu
`captures/tests_aws_serverless.txt`	Session AWS finale, Serverless
`captures/tests_aws_vm.txt`	Session AWS finale, VM
`captures/tests_aws_hybrid.txt`	Session AWS finale, Hybride
`captures/tests_local_serverless.txt`	Stack locale, Serverless (second passage sans réinitialisation)
`captures/tests_aws_premier_passage.txt`	Session AWS finale, premier passage du serverless et de l'hybride (scénario 4 de la comparaison, échecs expliqués ci-dessous)
`captures/tests_local_vm.txt`	Stack locale, VM (second passage sans réinitialisation)
`captures/tests_local_hybrid.txt`	Stack locale, Hybride (second passage sans réinitialisation)

Sur AWS, un premier passage du serverless et de l'hybride a échoué à deux scénarios : le 15 (délai réseau dépassé sur le poste de test ; la Lambda `api` n'a jamais dépassé 1,4 s) et le 35 (la vérification X-Ray cherchait un segment au nom de la machine Step Functions, que le service n'émet pas : elle vérifie désormais que la trace de l'exécution traverse la machine, étape par étape). Les deux suites ont été rejouées en entier ; dans celle du serverless, le scénario 35 a été interrompu par une coupure réseau du poste puis rejoué seul (scénarios 32 à 36, en fin de fichier).

D.3 Test de charge k6

Script réutilisable : `scripts/load/k6_shop.js`, lancé par `scripts/load_test.sh` (image Docker `grafana/k6`, rien à installer). Le parcours ne fait que des lectures (catalogue, recherche, fiche, avis, suivi, santé) : aucune commande ni aucun paiement n'est créé. Les solutions sont testées l'une après l'autre pour ne pas se partager le quota Lambda pendant la mesure ; le nombre d'utilisateurs virtuels est plafonné à 8 pour la même raison.

p95 par page (ms) :

Solution	catalogue	recherche	fiche	avis	suivi	sante
Serverless	309	332	193	162	225	153
VM	266	251	261	199	257	101
Hybride	303	321	163	155	158	156

Sorties : `captures/k6_resume.md`, `captures/k6_aws.txt` (sortie de k6) et le résumé JSON de chaque solution : `captures/k6_serverless.json`, `captures/k6_vm.json`, `captures/k6_hybrid.json`.

Lecture des résultats. À 8 utilisateurs virtuels (environ 14 requêtes par seconde, le rythme étant fixé par les pauses du parcours), les trois solutions répondent à 100 % sans aucun refus : une requête Lambda dure environ 0,1 s, soit à peine 1 à 2 exécutions simultanées en moyenne, loin du quota de 10. Les médianes (47 à 56 ms) et les p95 (241 à 272 ms) sont du même ordre : à ce débit, la latence est surtout celle du réseau et de CloudFront. La VM a la p95 la plus basse (pas de démarrage à froid) ; les maximums du serverless (514 ms) et de l'hybride (1,1 s) sont des démarrages à froid de Lambda. Le contraste avec le scénario 4 est net : 23 commandes envoyées **au même instant** dépassent le quota et le serverless en refuse 13 (429), alors qu'un trafic soutenu mais régulier passe sans refus. Le quota limite les rafales, pas le débit moyen. Le second compte utilisé pour la session finale a le même quota (10) que le premier : ces chiffres sont comparables à ceux des sessions précédentes.

Annexe E. Déroulement du projet, bugs trouvés et incident CloudTrail

E.1 Déroulement

1. **Version initiale** (commit de référence) : réseau et solution VM, données, bus d'événements et audit, serverless, supervision et sécurité, stack locale Docker, puis solution hybride et extension de la stack locale aux trois solutions.
2. **Lots fonctionnels**, chacun développé et testé en local, puis validé sur AWS dans une seule session (`up.sh`, tests, `down.sh`, vérification que le compte est vide), et enregistré par un commit de sauvegarde (regroupés à la fin en un seul commit) :

Lot	Fonctionnalités
0	Code commun regroupé dans `src/shop` (une seule implémentation des règles)
1	Stock par entrepôt, back-office, mouvements, annulation, réservations, alertes, CSV, valorisation
2a	Comptes Cognito, recherche et filtres, codes promo, avis, liste de souhaits
2b	Images produit (miniatures), suivi en temps réel par WebSocket
3a	Paiement Stripe, frais de port, workflow jusqu'à la livraison, e-mails SES, facture PDF
3b	Retours et remboursements, réapprovisionnement fournisseur
4	Tableau de bord, rapport quotidien, Athena + Glue, X-Ray, GitHub Actions, Grafana, k6
3. **Finalisation** : suite complète deux fois en local, session AWS finale avec k6 et	
captures, rapport, PDF, regroupement des commits. Une première session finale a	
échoué faute de quota SES (ci-dessous) ; elle a été refaite, après correction, sur un	
second compte AWS aux quotas identiques (10 Lambdas simultanées, SES en bac à sable),	
dont les ressources préexistantes (d'un autre projet) n'ont pas été touchées.	

E.2 Bugs trouvés par les tests (version initiale)

1. Pool de connexions épuisé (solution VM).

- **Symptôme :** au deuxième passage du scénario 4 en local, 3 requêtes sur 20 ont renvoyé `500 connection pool exhausted`.
- **Cause :** `psycopg2.pool.ThreadedConnectionPool` lève une erreur immédiatement quand ses 10 connexions sont prises, au lieu de faire attendre. Or le serveur HTTP crée un thread par requête.
- **Correction :** un `threading.BoundedSemaphore` de 10 places devant le pool. Une requête attend qu'une connexion se libère, jusqu'à 10 s. Au-delà, elle reçoit un `503` explicite au lieu d'un `500`.
- **Vérification :** 3 passages consécutifs du scénario 4 sans erreur.

2. Commande bloquée en `PACKED` (les trois solutions : la VM, et le code du processor commun au serverless et à l'hybride).

- **Symptôme possible :** une commande reste en `PACKED` et n'est jamais expédiée.
- **Scénario :** le traitement met la commande à jour, puis échoue en publiant l'événement suivant sur SNS (erreur réseau, timeout). SQS relivre le message, mais la mise à jour conditionnelle échoue, puisque le statut est *déjà* le statut cible. Le code concluait « déjà traité » et ignorait le message, si bien que l'événement suivant n'était jamais publié.
- **Correction** (`advance()` dans `src/vm/app.py` et `src/lambda/processor/handler.py`) : si la mise à jour échoue, on relit la commande en lecture cohérente.
- Si son statut est déjà le statut cible, on **republie** l'événement suivant.
- S'il est plus avancé, c'est un vrai doublon, qu'on ignore.

- **Droit IAM ajouté :** `dynamodb:GetItem`, pour la relecture côté Lambda.
- **Vérification :**
- en local : commande forcée en `PACKED`, puis `ORDER_CREATED` rejoué, elle atteint bien `SHIPPED` ;
- côté Lambda : un test sur moto couvre les 3 cas (normal, rejoué, vrai doublon).
- **Compromis assumé :** en livraison *at-least-once*, l'e-mail d'expédition peut exceptionnellement partir deux fois. C'est préférable à une commande bloquée.

3. Environ 20 s de latence à chaque étape traitée par Lambda (toutes les solutions).

- **Symptôme :** au premier test de l'hybride, le scénario 2 est resté bloqué en `PACKED` après 90 s, et le stock projeté n'avait pas rattrapé RDS au bout de 20 s. Une commande isolée devenait lisible entre 2,8 et 20,7 s après sa création, même sans aucune charge. Le serverless mettait déjà 27 à 51 s pour atteindre `SHIPPED`, contre 11 s pour la VM.
- **Diagnostic, par élimination :**
- **Le cache CloudFront des erreurs ?** Non : réponses `x-cache: Miss` ou `Error`, et la commande apparaissait au même moment en lisant DynamoDB directement.
- **Le quota Lambda ?** Non : la métrique `Throttles` était à 0 pour les Lambdas hybrides.
- **Instrumentation du projecteur :** il journalise désormais les horodatages SQS de chaque message.
  - De la publication à l'arrivée dans SQS : 0,1 à 0,8 s.
  - Prise du message par le poller Lambda : **5 ms**.
  - Appel de la fonction : environ **20 s plus tard**.
- **Cause :** les déclencheurs SQS → Lambda avaient une fenêtre de regroupement (`maximum_batching_window_in_seconds`) de 1 s. Avec une fenêtre non nulle, le poller continue son *long polling* de 20 s pour tenter de compléter le lot avant d'appeler la fonction.
- **Correction :** une fenêtre à 0 pour les 4 déclencheurs.
- **Vérification :**
- une commande hybride est lisible en **0,3 à 1,1 s** (5 mesures) au lieu de 3 à 21 s ;
- le serverless atteint `SHIPPED` en 15 s au lieu de 27 à 51 s.

Ce bug existait avant la solution hybride, mais seule la cohérence à terme l'a rendu visible.

4. Course entre la projection et le traitement (hybride).

- **Symptôme :** une commande mettait plus d'une minute à passer en `PACKED`.
- **Cause :** l'événement `ORDER_CREATED` est consommé par deux Lambdas : le projecteur, qui crée la commande dans DynamoDB, et le processor, qui la fait avancer. Quand le processor passait le premier, il ne trouvait pas la commande. Il levait alors une erreur, et SQS ne lui renvoyait le message qu'après le délai de visibilité de 60 s.
- **Correction :** le premier des deux consommateurs qui reçoit l'événement crée la commande (écriture conditionnelle `attribute_not_exists`, idempotente). Ni l'un ni l'autre n'attend plus l'autre.
- **Droit IAM ajouté :** `dynamodb:PutItem` pour `hybrid-processor`, sur sa seule table.
- **Vérification :** un test sur moto couvre 5 cas :
- processor avant la projection ;
- projection dans le désordre ;
- rejeu ;
- projection tardive qui n'écrase pas le statut ;
- processor après la projection.

5. Suivi figé après une erreur (front).

- **Cause :** dans le front, une erreur pendant le suivi (le `catch`) ne programmait pas de nouvel essai. Un 404 juste après la création (hybride) ou une coupure réseau arrêtait donc définitivement le rafraîchissement.
- **Correction :** en cas d'erreur, le front garde le dernier état affiché (d'abord la réponse du POST) et réessaie au tour suivant.

E.3 Bugs trouvés pendant les lots

Lot	Symptôme	Cause et correction
2b	Création de compte de test en échec quand le mot de passe aléatoire commençait par « - »	L'AWS CLI le lisait comme une option : `--password=<valeur>` ; le compte est enregistré pour nettoyage avant de fixer le mot de passe

Lot	Symptôme	Cause et correction
2b	Scénario 21 : original d'image « public » sur la VM	C'était la page `index.html` renvoyée par défaut ; le test compare désormais le contenu
3a	E-mails en échec (« Type non sérialisable float »)	Montants relus en `Decimal(str(…))` dans `shop/mail.py`
3a	Paiement en 409 sous concurrence (serverless)	Conflit de transaction DynamoDB traité comme refus : boucle de nouvel essai dans `mark_paid`
3a	Scénarios hybrides en échec juste après la création	Commande pas encore projetée : le test attend la projection
3b	Scénario 10 (réservations) en échec	Le réappro automatique ajoutait du stock pendant le test : produits de test sans seuil d'alerte
3b	Scénario 12 dépendant d'un lancement précédent	Identifiants de produit limités à 900 valeurs par préfixe : format élargi et identifiants aléatoires uniques (`scripts/testids.py`)
4	Rapport quotidien sans e-mail en local	L'initialisation locale ne mettait pas à jour le filtre d'un abonnement SNS existant
4	Validation finale locale : erreurs 500 et 502 en série au milieu des tests	Le noyau de la VM Docker tuait LocalStack faute de mémoire (OOM) : une trentaine de conteneurs Lambda restaient gardés au chaud 10 min. `LAMBDA_KEEPALIVE_MS` réduit à 1 min
Final	Session AWS : 429 en cascade sur les trois solutions (API, lecture hybride, e-mails, miniatures, WebSocket)	Quota SES du bac à sable dépassé (200 e-mails par 24 h, 1 par seconde) : chaque envoi refusé était réessayé et la Lambda `mailer` gardait ~7 s une des 10 exécutions du compte. Corrections : au plus 2 exécutions simultanées par file SQS (`maximum_concurrency`), nouveaux essais SES limités, et aucun envoi vers les domaines réservés (`example.com`) des commandes de test
Final	`up.sh` interrompu : `AddPermission` en 409 (« concurrent update operation ») sur la Function URL	Mise à jour de la Lambda encore en cours côté AWS : erreur transitoire, un second `terraform apply` dans la même session a terminé le déploiement
4	Alerte de retour en stock sans e-mail	L'adresse du client n'était pas enregistrée avec le souhait : ajoutée, et vérifiée par le scénario 28

E.4 Incident : double `terraform apply`, diagnostiqué avec CloudTrail

- **Symptôme :** un `./up.sh` échouait avec des dizaines d'erreurs `AlreadyExists` (DynamoDB, ALB, IAM, CloudTrail, CloudFront…). Or l'état Terraform local ne connaissait pas ces ressources.
- **Diagnostic :**
- Les événements CloudTrail (`lookup-events`), filtrés par utilisateur, adresse IP et agent `Terraform/1.16.4`, ont montré **deux `apply` lancés à 26 secondes d'intervalle**, le 27/10 à 08:56:50 et 08:57:16 UTC, depuis la même machine.
- Chaque `apply` avait tiré un suffixe aléatoire différent pour ses buckets (`25618d` et `fcdb04`).
- Le second avait écrit l'état local. Le premier avait créé sa partie, puis son état avait été écrasé. Ses ressources étaient donc **orphelines** : elles existaient dans le compte, mais aucun état ne les connaissait.
- **Remise en ordre, sans rien supprimer à l'aveugle :** 1. `./down.sh` pour détruire le déploiement connu de l'état (`fcdb04`) : 53 ressources ; 2. un script dédié, `cleanup_orphans.sh`, pour les orphelins `25618d` :
  - par défaut, il ne fait qu'une simulation ;
  - il ignore tout ce qui figure dans l'état ;
  - il supprime RDS sans snapshot final et vide aussi les versions des buckets.

  Après validation de la simulation, il a supprimé 14 ressources, dont une **base RDS et un ALB facturés en double**. 3. un inventaire en lecture seule, service par service, pour confirmer que le compte était vide. - **Suites :** au redéploiement, deux restes non visibles dans cet inventaire sont apparus, une *cache policy* et un *Origin Access Control* CloudFront, que le script recherche désormais. Un refus d'accès au rôle `AWSServiceRoleForAutoScaling` est aussi apparu. CloudTrail a montré que ce rôle venait d'être créé pour la première fois par AWS à la création de l'ASG ; le refus était un délai de propagation, et un nouvel `apply` l'a résolu. - **Leçons :** 1. **un état Terraform local ne protège pas contre deux exécutions simultanées**. Il faut un backend partagé avec verrou (§7) ; 2. CloudTrail est l'outil qui répond à « qui a créé quoi, quand et d'où ».

E.5 Deux services sur une `t3.micro` : mémoire et connexions RDS

Relevés faits pendant le test de charge k6 de la session finale (solutions VM puis hybride), par un petit script envoyé sur l'instance par **SSM Run Command** (sans SSH) : mémoire de l'instance et de chaque service systemd (cgroup), connexions PostgreSQL par base (`pg_stat_activity`), toutes les 2 s (150 relevés). Détail : `captures/ec2_rds_ressources.txt` ; métriques CloudWatch : annexe C.

Mesure	Résultat
Mémoire de l'instance	507 à 511 Mo utilisés sur 913 Mo, swap 0 Mo
Service `shop` (VM, port 8080)	72 à 73 Mo
Service `shop-hybrid` (port 8081)	79 à 80 Mo
Connexions PostgreSQL par base	`(système)` 5 à 5, `postgres` 1 à 1, `rdsadmin` 2 à 2, `shop` 1 à 7, `shop_hybrid` 1 à 1
Limite `max_connections` de `db.t3.micro`	81

Lecture : un second service sur la même `t3.micro` coûte une soixantaine de Mo, et la mémoire garde une large marge. Les connexions sont bornées par les pools (10 par service) : 2 services × 2 instances au plus × 10 = 40 connexions, sous la limite de 81. Au-delà (plus d'instances ou de services), il faudrait réduire les pools ou passer par RDS Proxy.

Annexe F. Coûts et sécurité en détail

F.1 Coût par service

Service	Tarif public (eu-west-3, ordre de grandeur)	Usage du projet	Coût
ALB	≈ 0,025 $/h + LCU	1, partagé VM et hybride	≈ 0,025 $/h
EC2 `t3.micro`	≈ 0,012 $/h	1 instance (2 au plus)	≈ 0,012 $/h
RDS `db.t3.micro`	≈ 0,02 $/h	1 instance, 2 bases	≈ 0,02 $/h
IPv4 publiques	0,005 $/h par adresse	ALB (2 AZ) + EC2	≈ 0,015 $/h
Lambda	1 M de requêtes et 400 000 Go-s gratuits par mois, à vie	quelques milliers d'appels	0 $
DynamoDB (à la demande, `PAY_PER_REQUEST`)	facturé à la requête (de l'ordre du dollar par million d'écritures, bien moins pour les lectures) ; 25 Go de stockage gratuits	3 tables, quelques milliers de requêtes par session	quelques centimes au plus
SNS, SQS	1 M de publications / requêtes gratuites par mois	quelques milliers	0 $
S3	0,024 $/Go-mois ; requêtes à l'unité	quelques Mo	< 0,01 $
CloudFront	1 To et 10 M de requêtes gratuits par mois	quelques Mo	0 $
Step Functions (Standard)	4 000 transitions gratuites par mois, puis 0,025 $ / 1 000	9 transitions par commande	0 $
EventBridge Scheduler	14 M d'appels gratuits par mois	3 par jour	0 $
SES	0,10 $ / 1 000 e-mails	quelques dizaines	< 0,01 $
Cognito	10 000 utilisateurs actifs gratuits par mois	comptes de test	0 $
API Gateway WebSocket	1 $ / M de messages, 0,25 $ / M de minutes de connexion	quelques centaines	< 0,01 $
Athena	5 $ / To lu, 10 Mo au minimum par requête	quelques requêtes	≈ 0 $
Glue Data Catalog	1 M d'objets gratuits	1 base, 1 table	0 $
X-Ray	100 000 traces gratuites par mois	quelques milliers	0 $
CloudWatch	10 alarmes gratuites, puis 0,10 $/mois chacune	une vingtaine, au prorata des heures	quelques centimes
CloudTrail	Premier trail de gestion gratuit	1 trail	0 $ (stockage S3 minime)

Les montants horaires sont ceux du prix public, payés ici par les crédits du plan gratuit ; ils sont à confirmer sur la page de prix de chaque service.

F.2 Adaptation au Free Tier

Élément	Choix par défaut	Raison	Pour l'activer
MSK (Kafka)	désactivé	Pas d'offre gratuite, ≈ 0,10 $/h par broker, 20 à 30 min de création ; le code est prêt	`enable_msk = true`
API Gateway REST	Function URL derrière CloudFront	Gratuite et suffisante	—
NAT Gateway	aucune	≈ 0,05 $/h ; EC2 en sous-réseau public fermé par security group, endpoints *gateway* S3 et DynamoDB gratuits	—
RDS Multi-AZ	Single-AZ	Double le prix	`db_multi_az = true`
2e instance EC2 permanente	`vm_min_size = 1` (2 au plus)	Une seule instance couverte	`vm_min_size = 2`
Clé KMS client	clés gérées par AWS	1 $/mois	`enable_kms_cmk = true`
Domaine	URL `*.cloudfront.net`	Zone Route 53 : 0,50 $/mois	`route53_zone_name`

Élément	Choix par défaut	Raison	Pour l'activer
Logs CloudWatch	3 jours de rétention	Stockage limité	`log_retention_days`
Athena	requête annulée au-delà de 100 Mo, résultats supprimés après 1 jour	Garde-fou de coût	—
Glue	projection de partitions, pas de crawler	Un crawler coûte ≈ 0,07 $ par passage	—

Le plan gratuit à crédits. Le compte a été créé le 22 octobre 2026. Il dépend donc du **nouveau Free Tier**, en vigueur depuis juillet 2025 :

- **Le plan gratuit dure 6 mois** et donne des **crédits** en dollars : 100 $ à l'inscription, et jusqu'à 100 $ de plus en explorant certains services. Ces crédits remplacent les anciens « 12 mois gratuits » par service (750 h EC2, 750 h RDS, etc.). Ce qui n'est pas gratuit à vie (ALB, EC2, RDS, IPv4 publiques) est payé avec ces crédits.
- **Les offres « toujours gratuites » restent valables** : 1 M de requêtes Lambda, 25 Go de stockage DynamoDB, 1 M de publications SNS, 1 M de requêtes SQS, 1 To CloudFront, etc.
- **État du compte**, lu avec `aws freetier get-account-plan-state` : plan `FREE`, statut `ACTIVE`, **179,97 $ de crédits restants**, expiration le **22/04/2027**.
- **Conséquence pour le projet :** les ressources « payantes » des solutions VM et hybride (ALB, EC2, RDS) ne facturent rien tant que les crédits durent. Le `down` systématique reste la règle, pour ne pas les consommer inutilement.
- **Limite des comptes récents :** ce plan applique un quota de **10 exécutions Lambda simultanées**, partagé par toutes les Lambdas du compte. Cela a une conséquence directe sur le test de charge (§6), et c'est l'une des raisons de la solution hybride.

F.3 Sécurité en détail

Responsabilité partagée. AWS est responsable de la sécurité *du* cloud (datacenters, matériel, hyperviseur, et OS des services managés : Lambda, RDS, DynamoDB…). Nous sommes responsables de la sécurité *dans* le cloud : identités, réseau, données, OS des EC2, code.

Rôles IAM (un par Lambda et un pour les EC2, ARN précis) :

Rôle	Droits principaux
Lambda `api`	DynamoDB (lecture, écriture, transaction) sur sa table ; `sns:Publish` ; S3 sur `exports/ imports/ invoices/ reports/` et `originals/` ; lecture du seul paramètre SSM Stripe
Lambda `hybrid-read`	`dynamodb:GetItem/Query` sur la table hybride : **aucune écriture** (et `READ_ONLY` dans le code)
Lambdas `processor`	`states:StartExecution` sur leurs machines ; lecture de **leur** file
Lambdas `fulfillment`, `supplier`	`GetItem/UpdateItem` (et `PutItem/DeleteItem` pour les bons) sur leur table ; `sns:Publish`
Lambda `audit`	`s3:PutObject` sur `events/*` uniquement
Lambda `mailer`	`ses:SendRawEmail` sur la seule identité vérifiée
Lambdas `reporter`	lecture de leur table ; `s3:PutObject` sur `reports/*` ; `sns:Publish`
EC2	lecture du code dans S3, des seuls paramètres SSM utiles, de **ses** files ; `sns:Publish` ; écritures conditionnelles sur la table hybride ; Session Manager
Step Functions, Scheduler	`lambda:InvokeFunction` (et `sqs:SendMessage` pour la VM) sur leurs seules cibles ; X-Ray

Toutes les Lambdas ont aussi `xray:PutTraceSegments` (pas d'ARN de ressource pour cette action).

Jetons. La vérification est faite dans `shop/auth.py`, sans dépendance : RS256 uniquement (jamais `none` ni HS256), signature RSA vérifiée avec la clé du JWKS désignée par le `kid`, émetteur, expiration (60 s de tolérance), client (`aud` ou `client_id`) et `token_use`. Seul un émetteur `cognito-idp.<région>.amazonaws.com` est accepté, sauf avec `ALLOW_LOCAL_AUTH=1`, défini uniquement dans la stack Docker.

Paiement. Les données de carte ne passent que par Stripe.js (SAQ A). Le `client_secret` n'est renvoyé qu'à la création de la commande ; `confirm-payment` relit le PaymentIntent (statut, montant, devise, numéro de commande) ; les clés d'idempotence évitent tout double paiement ou double remboursement. La clé secrète est dans SSM, lue à l'exécution ; seules les clés `pk_test_` et `sk_test_` sont acceptées par Terraform.

Réseau. CloudFront ajoute un en-tête secret (`X-Origin-Verify`) vérifié par la Lambda et par l'ALB : appeler la Function URL ou l'ALB directement renvoie 403. La règle hybride de l'ALB exige aussi `X-Shop: hybrid`. L'ALB n'accepte que la *prefix list* CloudFront ; RDS n'accepte que le security group des EC2.

Chiffrement. Au repos : DynamoDB, RDS, SNS, SQS, S3, SSM SecureString (clé client KMS en option). En transit : HTTPS imposé par CloudFront, TLS pour tous les appels AWS et Stripe.

Traçabilité. CloudTrail (validation d'intégrité, 30 jours), logs CloudWatch, alarmes (DLQ non vide, erreurs Lambda, cibles ALB), traces X-Ray, archive d'audit des événements.

Annexe G. Présenté mais non implémenté

Ces services ont été étudiés pour la boutique mais pas déployés (coût, délai, ou hors périmètre du sujet). Chaque ligne indique où ils se brancheraient.

Service	Usage pour Threadly	Intégration envisagée	Pourquoi pas ici
Webhook Stripe (production)	Confirmer le paiement dès qu'il réussit	Point d'entrée `POST /api/stripe/webhook` vérifiant la signature (`Stripe-Signature`, secret dans SSM), qui appelle la même confirmation idempotente que `confirm-payment`	Point d'entrée public et secret de plus ; la purge suffit en mode test
Amazon Bedrock	Rédaction des descriptions produit, assistant de recherche en langage naturel	Lambda appelant un modèle de langage de Bedrock depuis le back-office ; garde-fous Bedrock Guardrails	Facturé au jeton, pas d'offre gratuite
Amazon Personalize	Recommandations « vous aimerez aussi »	Import des événements d'audit (S3) comme interactions ; campagne interrogée par l'API	Entraînement et campagne facturés à l'heure
Amazon Forecast	Prévision de la demande pour régler les seuils de réappro	Historique des ventes (Athena) → prévisions → seuils par produit. Forecast n'est plus ouvert aux nouveaux clients : SageMaker Canvas en est le remplaçant	Service fermé aux nouveaux comptes ; coût d'entraînement
AWS WAF	Protection contre les bots, l'injection, les abus de l'API	Web ACL sur les distributions CloudFront (règles gérées, limitation de débit)	≈ 5 $/mois par ACL + règles
Amazon QuickSight	Tableaux de bord métier pour la direction	Source Athena (table `events`)	Abonnement par utilisateur
AWS Secrets Manager	Secrets avec rotation automatique (mot de passe RDS)	Remplace SSM SecureString ; rotation par Lambda	0,40 $/mois par secret ; SSM gratuit suffit
Amazon OpenSearch	Recherche plein texte tolérante aux fautes, facettes	Index alimenté par les événements `PRODUCT_CHANGED`	Domaine facturé à l'heure
Odoo / ERP	Comptabilité, achats, fournisseurs réels	Abonnement aux événements (`ORDER_PAID`, `PURCHASE_ORDER_CREATED`) par une file SQS dédiée ; API XML-RPC d'Odoo	Hors périmètre ; le réappro simule le fournisseur

Annexe H. Mode d'emploi pour le correcteur

H.1 Prérequis

- Terraform ≥ 1.9, AWS CLI configurée (`aws configure`, région `eu-west-3`) ;
- `python3`, `curl` ; Docker (stack locale, k6, actionlint).

H.2 En local, sans AWS ni coût

```
docker compose -f local/docker-compose.yml up --build -d   # environ 1 min au premier lancement
./scripts/test_scenario.sh http://localhost:8080           # VM
./scripts/test_scenario.sh http://localhost:8081           # serverless
./scripts/test_scenario.sh http://localhost:8082           # hybride
./scripts/load_test.sh                                     # k6 sur les trois
docker compose -f local/docker-compose.yml down -v         # tout arrêter et effacer
```

Adresse	Contenu
http://localhost:8080 · :8081 · :8082	Boutique VM, serverless, hybride (back-office : `/admin.html`)
http://localhost:3000	**Grafana** : tableau de bord « Threadly : ventes et stock » (bases VM et hybride, compte en lecture seule)
http://localhost:4566/_aws/ses	E-mails envoyés (LocalStack)
http://localhost:9100	Fournisseur d'identité local : `admin@threadly.local` / `admin-local-pass`, `magasinier@threadly.local` / `mag-local-pass`, `client@threadly.local` / `client-local-pass`
http://localhost:12111	Faux Stripe local (cartes `pm_card_visa`, `pm_card_chargeDeclined`)

LocalStack (édition communautaire 4.14.0) simule Lambda, DynamoDB, SNS, SQS, S3, SSM, Step Functions et SES avec le code exact de `src/` ; nginx joue CloudFront. Athena, Glue, X-Ray et EventBridge Scheduler n'y existent pas : le rapport s'y déclenche à la demande.

H.3 Sur AWS

1. **Configuration** : `cp infra/terraform.tfvars.example infra/terraform.tfvars`, puis renseigner `notification_email` et le **paiement** : - `payment_mode = "stripe"` (par défaut) avec les clés **test** du tableau de bord Stripe (`stripe_publishable_key = "pk_test_…"`, `stripe_secret_key = "sk_test_…"`) ; sans clés, `terraform plan` s'arrête avec un message clair ; - ou `payment_mode = "simulated"` sans compte Stripe (commandes confirmées aussitôt).
2. **Vérification SES (une seule fois)** : `bash aws sesv2 create-email-identity --region eu-west-3 --email-identity vous@example.com # cliquer sur le lien reçu, puis vérifier : aws sesv2 get-email-identity --region eu-west-3 --email-identity vous@example.com --query VerifiedForSendingStatus` `up.sh` refuse de déployer tant que l'identité n'est pas vérifiée ; `down.sh` ne la supprime jamais.
3. **Déploiement** : `./up.sh` (10 à 15 min, **un seul à la fois**), puis confirmer l'abonnement SNS reçu par e-mail (alarmes).
4. **Comptes** : créer les utilisateurs dans le pool Cognito et les ajouter aux groupes `admin` ou `magasinier` (les tests créent et suppriment leurs propres comptes).
5. **Tests** : `bash for u in serverless_url vm_url hybrid_url; do ./scripts/test_scenario.sh "$(terraform -chdir=infra output -raw $u)" done ./scripts/load_test.sh aws # k6, 8 utilisateurs virtuels au plus ./scripts/daily_report.sh serverless|vm|hybrid # rapport quotidien à la demande` Paiement de test dans le navigateur : carte `4242 4242 4242 4242`, date future, n'importe quel code.
6. **Suppression** : `./down.sh` (environ 10 min), puis `./scripts/cleanup_orphans.sh` (simulation) pour vérifier qu'il ne reste rien.

H.4 Intégration continue

`.github/workflows/ci.yml` : `terraform fmt -check` et `terraform validate`, puis la stack Docker et tous les scénarios, un job par solution en parallèle (runners ARM, comme les Lambdas). Aucun secret ni accès AWS n'est nécessaire.